

# File Backup Service

Dockerized FastAPI + S3 + RDS SQL Server -- AWS Region Migration Playbook

Singapore (ap-southeast-1) → Sydney (ap-southeast-2) | Prepared by Firoz Mahmud | July 10, 2026

## Contents

|    |                              |    |
|----|------------------------------|----|
| 1  | Executive Summary            | 1  |
| 2  | Architecture Overview        | 2  |
| 3  | AWS Resource Sizing          | 2  |
| 4  | Database Schema              | 4  |
| 5  | FastAPI Application          | 5  |
| 6  | Containerization             | 7  |
| 7  | Deployment Guide             | 8  |
| 8  | Region Migration Runbook     | 9  |
| 9  | Migration Automation Scripts | 11 |
| 10 | Security Notes               | 11 |
| 11 | Summary                      | 11 |

## 1 Executive Summary

This document specifies a proof-of-concept (POC) backend service, its AWS footprint, and a **region-migration playbook**: everything needed to move the whole stack — compute, database, object storage, including data — from **Singapore (ap-southeast-1)** to **Sydney (ap-southeast-2)** as concrete, runnable commands.

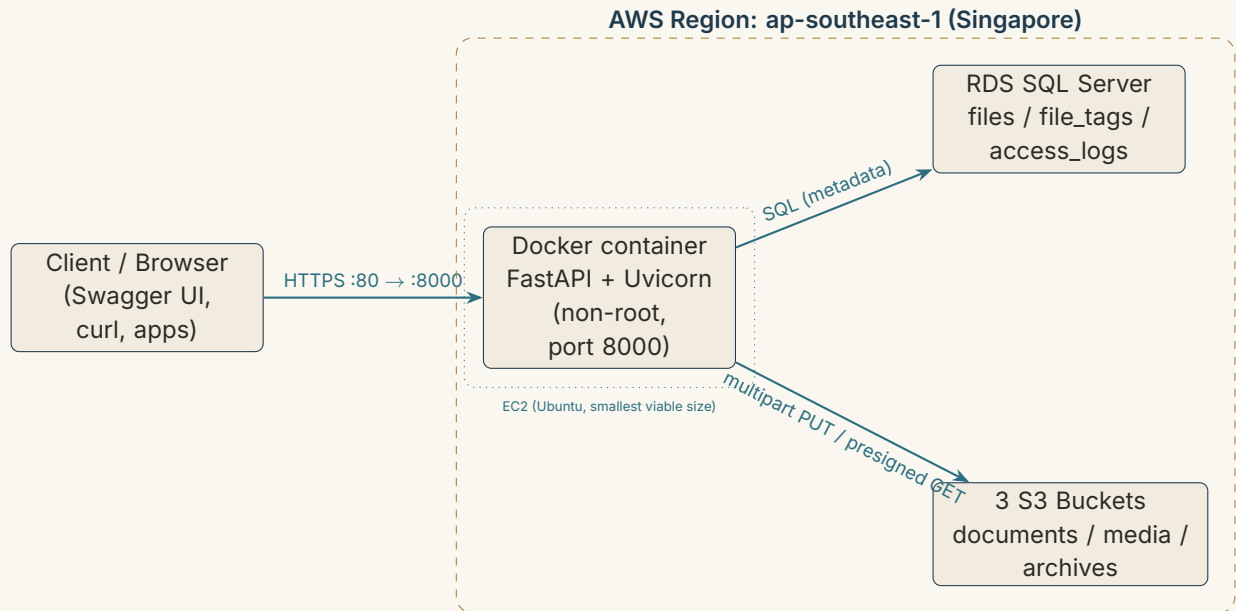
Users `POST` a file plus optional description/tags; it streams to one of **three S3 buckets** routed by category (documents/media/archives, auto-detected or explicit), never touching local disk. A SQL Server row records where it landed. Swagger UI (`/docs`) is auto-generated by FastAPI. The app runs as a hardened, non-root container on the smallest practical Ubuntu EC2 instance, port 80 forwarded to the container.

**What makes region migration painless:** every AWS-region-specific value lives in exactly one file, `app/config.py`. Migrating regions is a data-replication exercise plus an environment-variable change, not a code change (Section 8).

**i Note**

All code here was smoke-tested end to end against a real SQL Server instance and S3-compatible store, and separately as the built Docker image (non-root, read-only filesystem, capabilities dropped) – and is included directly from the source files, not retyped.

## 2 Architecture Overview



- **EC2** runs one thing: the Docker container (Section 6), port 80 → container 8000. Stateless – no file data touches the container’s own (read-only) filesystem, so migrating compute is just a new instance running the same image.
- **RDS SQL Server** holds only metadata: three tables (Section 4).
- **Three S3 buckets** hold the file bytes; the app never returns S3 URLs directly, only short-lived presigned URLs.
- RDS and all buckets sit behind a private VPC / bucket policy, public access blocked.

## 3 AWS Resource Sizing (Cost-Effective POC → Production Path)

### ! Important

Prices are **indicative on-demand, ap-southeast-1**, ±15–20 % – confirm exact numbers via the [AWS Pricing Calculator](#) before committing budget.

### 3.1 EC2 --- Application Server (Ubuntu, Docker)

| Attribute                  | This POC (as deployed)                                                                                                                                                                   |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Instance type              | <b>t2.micro</b> (1 vCPU burstable, 1 GiB RAM) – lowest-cost general-purpose type; switch to t3.micro (similar price, 2 vCPU, better burst credits) only if you hit CPU-credit exhaustion |
| AMI                        | Ubuntu Server 22.04/24.04 LTS                                                                                                                                                            |
| Root volume                | <b>10 GiB gp3</b> EBS – app image is ≈145 MB; this is Docker Engine + layer cache headroom, not app-driven                                                                               |
| Est. cost (ap-southeast-1) | ≈ \$8.5/mo on-demand                                                                                                                                                                     |

**IAM instance role** (not static keys) scoped to `s3:PutObject, GetObject, DeleteObject, AbortMultipartUpload` on all three bucket ARNs (policy in Section 7). Security group: port 22 from admin IP only, port 80 from whatever needs Swagger/API access.

## 3.2 RDS --- SQL Server

### ! Important

**Edition drives cost, not instance size.** Express is capped at 10 GiB/database (too small). Standard/Enterprise cost far more. **Web Edition:** no size cap, well below Standard, license permits this public-facing use case – cheapest edition that fits and is compliant.

| Attribute                  | Recommendation                                                                                                                                       |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Engine edition             | <b>SQL Server 2019, Web Edition</b>                                                                                                                  |
| Instance class             | <b>db.t3.small</b> (as deployed) – verify orderable via <code>aws rds describe-orderable-db-instance-options --engine sqlserver-web</code>           |
| Storage                    | <b>20 GiB gp3</b> (as deployed)                                                                                                                      |
| Multi-AZ                   | <b>Disabled</b> for POC; enable before production                                                                                                    |
| Public accessibility       | <b>Disabled</b> — reachable only from the app's security group                                                                                       |
| Est. cost (ap-southeast-1) | <b>\$109.34/mo</b> (actual AWS console quote: \$106.58 instance + \$2.76 storage, backups/encryption off) – Microsoft licensing, not a sizing choice |

## 3.3 S3 --- Three Buckets, One Per Category

Every upload routes into one of three buckets by category (`documents/media/archives`), auto-detected or explicit (Section 5.4). Splitting storage this way costs nothing extra (billed per object, not per bucket) and buys tighter IAM scoping and blast-radius isolation later.

| Attribute                   | Recommendation (all three buckets)                                                                                                                                                     |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Storage class               | <b>S3 Standard</b>                                                                                                                                                                     |
| Versioning                  | <b>Disabled</b> – not sensitive data, and versioning has a real storage-cost tail (old versions keep billing); turn on later if accidental-overwrite protection becomes worth the cost |
| Encryption                  | SSE-S3 (AES-256), applied automatically by AWS at no cost – not something to configure or pay extra for                                                                                |
| Public access               | Fully blocked; all reads go through short-lived presigned URLs                                                                                                                         |
| Est. cost @ 100 GB combined | ≈\$2.5/mo, scales with data not bucket count                                                                                                                                           |

### 3.4 Monthly Cost Summary

| Component                                    | POC scale                         | Prod-like (20 GB RDS / 100 GB S3) | Notes                                                                                          |
|----------------------------------------------|-----------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------|
| EC2 t2.micro (Docker)                        | \$8.4/mo                          | \$8.4/mo                          | Compute is workload-driven, not data-volume-driven                                             |
| RDS SQL Server Web, db.t3.small + 20 GiB gp3 | <b>\$109.34/mo</b> (actual quote) | \$109.34/mo                       | License-inclusive rate dominates the bill                                                      |
| S3 Standard (3 buckets)                      | <\$1/mo at a few GB               | ≈\$2.5/mo at 100 GB combined      | Scales with total data, not bucket count                                                       |
| <b>Total (on-demand)</b>                     | ≈ <b>\$118/mo</b>                 | ≈ <b>\$120/mo</b>                 | Dominated by the SQL Server license; PostgreSQL would put this in the \$35–40/mo range instead |

**Region cost note:** Sydney (ap-southeast-2) on-demand rates run ≈15–25 % above Singapore for EC2/RDS, directionally – re-run the Pricing Calculator for the exact rate before migrating.

## 4 Database Schema

A `files` row owns zero-or-more `file_tags` rows (metadata) and `access_logs` rows (audit trail), both `ON DELETE CASCADE`.

### i Note

Two SQL Server fixes caught by testing against a real container: UUID columns use SQLAlchemy's portable `Uuid` type (maps to `UNIQUEIDENTIFIER`, not the Postgres-only `dialects.postgresql.UUID`); and `s3_key` is capped at 700 chars, not S3's 1024 limit, because SQL Server rejects a `UNIQUE` index over 900 bytes.

## 4.1 files

| Column                   | Type                 | Purpose                                                                                                                                                              |
|--------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id                       | Uuid, PK             | Primary key, generated client-side (uuid4); UNIQUEIDENTIFIER on SQL Server                                                                                           |
| original_filename        | varchar(512)         | As uploaded by the client                                                                                                                                            |
| s3_key                   | varchar(700), unique | Object key inside the bucket, e.g. uploads/{uuid}/{filename} – capped for SQL Server's unique-index byte limit, see above                                            |
| s3_bucket                | varchar(255)         | Which of the three buckets the object lives in (recorded per-row, not just derived from category, so historical rows remain correct after a bucket/region migration) |
| category                 | varchar(32)          | documents / media / archives – which bucket this file was routed to, filterable via GET /api/v1/files?category=                                                      |
| content_type             | varchar(255)         | MIME type reported at upload time                                                                                                                                    |
| size_bytes               | bigint               | Computed server-side while streaming, not trusted from the client                                                                                                    |
| checksum_sha256          | varchar(64)          | Computed server-side while streaming to S3                                                                                                                           |
| description              | text, nullable       | Free-text metadata                                                                                                                                                   |
| is_deleted               | boolean              | Soft-delete flag; S3 object is removed but the DB row is retained for audit                                                                                          |
| uploaded_at / updated_at | timestamp            | Set/maintained by the ORM                                                                                                                                            |

## 4.2 file\_tags

| Column              | Type                                 | Purpose                                     |
|---------------------|--------------------------------------|---------------------------------------------|
| id                  | Uuid, PK                             | Primary key                                 |
| file_id             | Uuid, FK → files.id (cascade delete) | Owning file                                 |
| tag_key / tag_value | varchar(128) / varchar(512)          | Arbitrary metadata, e.g. department=finance |

## 4.3 access\_logs

| Column      | Type                                 | Purpose                           |
|-------------|--------------------------------------|-----------------------------------|
| id          | Uuid, PK                             | Primary key                       |
| file_id     | Uuid, FK → files.id (cascade delete) | Owning file                       |
| action      | varchar(32)                          | One of upload / download / delete |
| accessed_at | timestamp                            | When the action happened          |
| note        | varchar(255), nullable               | Optional free text                |

### i Note

Tables are created at startup via `Base.metadata.create_all()` (`app/main.py`) – fine for a POC; replace with Alembic migrations before production, since `create_all()` has no schema versioning.

## 5 FastAPI Application

All source lives in the repository; this section names the modules, classes, and functions rather than reprinting them.

## 5.1 Module Overview

| File                           | Contents                                                                                     |
|--------------------------------|----------------------------------------------------------------------------------------------|
| app/config.py                  | Settings class – <b>the only region-dependent file</b> (Section 5.2)                         |
| app/database.py                | SQLAlchemy engine/session setup                                                              |
| app/models.py                  | FileRecord, FileTag, AccessLog ORM classes (schema in Section 4)                             |
| app/schemas.py                 | Pydantic request/response models, UploadCategory enum                                        |
| app/crud.py                    | create_file_record(), get_file(), list_files(), soft_delete_file(), log_action(), get_logs() |
| app/s3_client.py               | upload_fileobj(), delete_object(), generate_presigned_download_url()                         |
| app/routers/files.py           | /api/v1/files/* endpoints, category routing (Section 5.4)                                    |
| app/main.py                    | FastAPI app, /health, /api/v1/region-info                                                    |
| scripts/*                      | Migration automation (Section 9)                                                             |
| Dockerfile, docker-compose.yml | Section 6                                                                                    |

## 5.2 Configuration --- the Region-Dependent File

Every other file reads **AWS region, bucket names, and database location exclusively through app/config.py's Settings class** – nothing else changes on a region migration. Key fields: `aws_region`, `s3_bucket_documents/_media/_archives`, `database_url`; method `bucket_for_category(category)` maps an upload category to its bucket name.

## 5.3 S3 Client and CRUD

`s3_client.upload_fileobj()` streams to S3 via multipart upload unconditionally (so the checksum-while-streaming logic is size-independent), returning `(sha256_checksum, size_bytes)`. `crud.get_file()` and `crud.list_files()` carry the real MSSQL portability fix noted in Section 5.6.

## 5.4 API Routes

In `upload_file()`: an explicit `category` field wins (422 if invalid); otherwise it's inferred from content type via `_infer_category()`. The resolved category picks the bucket through `bucket_for_category()`, written to `files.s3_bucket` so download/delete never re-derive it.

## 5.5 Dependencies

FastAPI, Uvicorn, SQLAlchemy, pymssql, boto3, pydantic-settings, python-multipart, python-dotenv – pinned versions in `requirements.txt`.

## 5.6 API Surface (as generated by Swagger / OpenAPI)

| Method | Path                             | Purpose                                                                                                                                                       |
|--------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST   | /api/v1/files/upload             | Upload a file (multipart form: file, category [optional], description, tags JSON)                                                                             |
| GET    | /api/v1/files                    | Paginated list, optional <code>?category=documents media archives</code> filter                                                                               |
| GET    | /api/v1/files/{file_id}          | Metadata + category + tags for one file                                                                                                                       |
| GET    | /api/v1/files/{file_id}/download | Returns a presigned URL against whichever bucket the file lives in, time-limited                                                                              |
| GET    | /api/v1/files/{file_id}/logs     | Access log for one file                                                                                                                                       |
| DELETE | /api/v1/files/{file_id}          | Soft-delete the row, hard-delete the S3 object from its bucket                                                                                                |
| GET    | /health                          | Liveness probe                                                                                                                                                |
| GET    | /api/v1/region-info              | Reports region, all three bucket names, and DB host this instance is currently bound to — the fastest way to confirm a migration cutover actually took effect |

Swagger UI is auto-generated at `/docs` (ReDoc at `/redoc`), zero extra code.

### i Note

Read back from the app's live OpenAPI schema during testing, not copied from a design doc. That same test run caught a real MSSQL bug: SQLAlchemy's `.is_(False)` renders `IS 0`, which SQL Server rejects – fixed with `== False` instead (app/crud.py above).

## 6 Containerization

Multi-stage build, distroless runtime (no shell, no package manager), non-root by default; compose adds read-only-filesystem and capability-dropping. Verified by actually building and running it (tip box at end of section), not asserted.

### 6.1 Dockerfile

Multi-stage (full source in `Dockerfile` in the repository). Builder installs deps with a full Debian userland, then is discarded – only packages + app code cross into the final image, `gcr.io/distroless/python3-debian12:nonroot` (no `/bin/sh`, confirmed by trying to exec one). No compiler needed in the builder: `pymssql` ships a prebuilt wheel with FreeTDS bundled.

### 6.2 docker-compose.yml

Three services (full source in `docker-compose.yml`): app, plus `db/db-init` standing in for RDS locally – production points `DATABASE_URL` at real RDS instead. app hardening: read-only rootfs (tmpfs for upload spooling), `cap_drop: ALL`, `no-new-privileges`, explicit non-root UID. Host port 80 → container 8000.

### i Note

db can't go distroless (no such SQL Server image exists) but runs non-root by default. `cap_drop: ALL` alone breaks it: `sqlservr` has `cap_net_bind_service` as a *file* capability, which the kernel won't grant at exec unless it's in the caller's bounding set – fixed by adding back just `NET_BIND_SERVICE`.

### 6.3 Environment Templates

`.env.example` is for running the app directly; `.env.docker.example` is what `docker compose up` reads for `${VAR}` substitution (not mounted into the container; `DATABASE_URL` is assembled from `MSSQL_*` + the db service name). Both templates are in the repository root.

**! Important**

Docker Compose prefers shell-exported env vars over `.env`, per-variable – **run** `env | grep AWS_` **before** `docker compose up` on shared machines, unset any hits (`env -u AWS_ACCESS_KEY_ID -u AWS_SECRET_ACCESS_KEY -u AWS_REGION docker compose up`).

## 6.4 Verification

| Check                                              | Result                                                                                                                                                                                                              |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No shell in the final image                        | <code>docker run --entrypoint /bin/sh &lt;image&gt;</code> → <code>stat /bin/sh: no such file or directory</code>                                                                                                   |
| Runs as non-root                                   | <code>os.getuid()</code> inside the container → 65532                                                                                                                                                               |
| Image size                                         | 145 MB (distroless base + app deps + code)                                                                                                                                                                          |
| Driver imports cleanly with no missing shared libs | <code>import pymssql; sqlalchemy.create_engine('mssql+pymssql://...')</code> succeeds inside the container with no apt packages installed                                                                           |
| Full request cycle through the hardened container  | upload → correct bucket per category → category-filtered list → download (byte-for-byte match) → delete → 404, all through host port 80, with <code>read_only: true</code> and <code>cap_drop: ALL</code> in effect |

## 7 Deployment Guide --- Source Region (Singapore)

### 7.1 One-Time Setup

1. **VPC:** `vpc-04d1607e143b9ee1d` (Singapore). EC2 and RDS in the same VPC.
2. **Security groups:** `dev-ssh-access-sg` (inbound 22 from admin IP), `dev-web-access-sg` (inbound 80 from API/Swagger consumers), `dev-mssql-access` (inbound 1433 from `dev-web-access-sg` only). EC2 gets both `dev-ssh-access-sg` and `dev-web-access-sg`.
3. **RDS:** `file-backup-db`, `db.t3.small` SQL Server 2019 **Web Edition**, 20 GiB `gp3`, Multi-AZ off, in `dev-mssql-access`, VPC `vpc-04d1607e143b9ee1d`. Master user `admin`. Endpoint: `file-backup-db.c9g4u6ekudby.ap-southeast-1.rds.amazonaws.com:1433`.

**! Important**

**As deployed, this instance is publicly accessible** (has a public IP), diverging from this document's recommendation of **Disabled**. Access is still restricted by `dev-mssql-access` (only `dev-web-access-sg` allowed in) – confirmed by testing: unreachable from an outside IP, times out at the TCP level. That security-group restriction is doing the job `public-access-off` would normally do; still, prefer **Disabled** for anything beyond this POC, since it removes a layer of exposure the SG rule alone doesn't (e.g. a future SG rule mistake).

4. **Create the database:** unlike MySQL/PostgreSQL on RDS, SQL Server has no "initial database name" option at creation time – connect after the instance shows `Available` and create it yourself:

```
1 sqlcmd -S file-backup-db.c9g4u6ekudby.ap-southeast-1.rds.amazonaws.com -U admin -P 'devpassword' -C -N \
2 -Q "CREATE DATABASE filebackup;"
```

5. **S3:** create all three buckets in `ap-southeast-1` – `myco-file-backup-documents-sg`, `-media-sg`, `-archives-sg` – each S3 Standard, public access fully blocked (versioning left off; see Section 3.3).
6. **IAM role:** create `ec2-file-backup-role`, policy scoped to `s3:PutObject`, `GetObject`, `DeleteObject`, `AbortMultipartUpload`, `ListMultipartUploadParts` on the three bucket ARNs only, never `s3:*/Resource:*`. Attach to the EC2 instance profile.
7. **EC2:** `dev-file-backup-server-ec2`, a `t2.micro` (10 GiB `gp3` root volume), **Ubuntu Server 22.04 or 24.04 LTS**, with `dev-ssh-access-sg` + `dev-web-access-sg` and the IAM role attached (key pair `dev-ssh-access-kp`).



## 7.2 On the EC2 Instance

```
1 curl -fsSL https://get.docker.com | sudo sh
2 sudo usermod -aG docker ubuntu
3 newgrp docker # or log out/in for the group change to take effect
4 docker --version
5 docker compose version
```

**Listing 1:** Install Docker on Ubuntu (official convenience script)

```
1 git clone <your-repo-url> /opt/file-backup
2 cd /opt/file-backup
3
4 cp .env.docker.example .env
5 # Edit .env: set AWS_REGION and the three S3_BUCKET_* names for this region.
6 # Leave AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY blank -- the IAM instance
7 # role supplies credentials automatically. Set MSSQL_SA_PASSWORD to a
8 # strong value if running the bundled db container; if pointing at real
9 # RDS instead, skip the db/db-init services (docker compose up app) and
10 # set DATABASE_URL directly as an override.
11
12 docker compose up -d --build
13 docker compose ps # app, db, db-init all healthy/exited-0
14 curl http://localhost/health
15 # Swagger UI: http://<ec2-public-ip>/docs
```

**Listing 2:** Deploy the app

### i Note

Port 80 → container 8000, Swagger reachable with no reverse-proxy step. Before real usage: TLS termination (Nginx/ALB) in front, dev-web-access-sg restricted to 443.

## 7.3 Pointing at Real RDS Instead of the Bundled db Container

Once RDS exists, set DATABASE\_URL in .env and bring up only app with --no-deps (otherwise Compose still starts db/db-init first, per their depends\_on):

```
1 echo 'DATABASE_URL=mssql+pymssql://admin:devpassword@file-backup-db.c9g4u6ekudby.ap-southeast-1.rds.amazonaws.com:1433/filebackup' >> .env
2 docker compose up -d --build --no-deps app
```

## 8 Region Migration Runbook: Singapore → Sydney

What changes on migration: AWS\_REGION, the three S3\_BUCKET\_\* names (buckets don't move between regions), DATABASE\_URL, and the EC2 instance itself (new instance, same image). Nothing else – app code, schema, and IAM policy shape are identical.

### 8.1 Region Reference

| City      | Region code    | Role             |
|-----------|----------------|------------------|
| Singapore | ap-southeast-1 | Source (current) |
| Sydney    | ap-southeast-2 | Migration target |

### 8.2 S3 Transfer Mechanics and Cost

aws s3 sync between two s3:// URIs is a server-side copy – bytes move directly between buckets over AWS's network, never through the invoking machine, so a temporary EC2 instance in the source region buys nothing here. Cost is AWS's inter-region rate ( $\approx$ \$0.02/GB, confirm exact rate in the Calculator) –  $\approx$ \$4 for 200 GB, same whether split across three buckets or one. Running the three bucket syncs **in parallel** (what s3\_migrate\_all\_buckets.sh does) is what actually saves wall-clock time.

### 8.3 Phase 0 --- Pre-Migration Preparation (source stays live, zero risk)

1. dev-ssh-access-kp, dev-ssh-access-sg, dev-web-access-sg, dev-mssql-access already exist in ap-southeast-2 (Sydney) – confirm a target VPC and a DB subnet group spanning  $\geq 2$  AZs exist too.
2. Create all three destination S3 buckets in one call:

```
1 ./scripts/s3_migrate_all_buckets.sh create-dest \
2   ap-southeast-1 sg ap-southeast-2 sydney
```

3. Create an IAM role in the target region identical in shape to ec2-file-backup-role, scoped to the three new bucket ARNs.
4. If the source RDS instance is encrypted, create a KMS key in the target region (KMS keys are regional and cannot be copied directly; the snapshot-copy step re-encrypts under a target-region key).

### 8.4 Phase 1 --- Warm Data Sync (source still live)

**S3** (all 3 buckets in parallel, repeatable):

```
1 ./scripts/s3_migrate_all_buckets.sh warm \
2   ap-southeast-1 sg ap-southeast-2 sydney
```

**RDS** (non-destructive snapshot copy + restore; source untouched):

```
1 python scripts/rds_region_migration.py \
2   --source-region ap-southeast-1 \
3   --target-region ap-southeast-2 \
4   --db-instance-identifier file-backup-db \
5   --target-db-instance-identifier file-backup-db-sydney \
6   --db-instance-class db.t3.small \
7   --subnet-group filebackup-subnet-group-sydney \
8   --security-group-ids sg-xxxxxxx
```

### 8.5 Phase 2 --- Cutover Window (brief write-pause)

1. Maintenance mode: docker compose stop app.
2. Final sync (catches anything since the warm pass):

```
1 ./scripts/s3_migrate_all_buckets.sh final \
2   ap-southeast-1 sg ap-southeast-2 sydney
```

3. One more RDS snapshot-copy-restore (same command as Phase 1).
4. Verify:

```
1 ./scripts/s3_migrate_all_buckets.sh verify \
2   ap-southeast-1 sg ap-southeast-2 sydney
```

### 8.6 Phase 3 --- Redeploy in the Target Region

1. Launch EC2 in ap-southeast-2 (same spec as Section 7: Ubuntu LTS, t2.micro), install Docker, deploy the identical code/image.
2. Edit .env:

```
1 AWS_REGION=ap-southeast-2
2 S3_BUCKET_DOCUMENTS=myco-file-backup-documents-sydney
3 S3_BUCKET_MEDIA=myco-file-backup-media-sydney
4 S3_BUCKET_ARCHIVES=myco-file-backup-archives-sydney
5 DATABASE_URL=mssql+pymssql://admin:devpassword@file-backup-db-sydney.xxxxx.ap-southeast-2.rds.amazonaws.com:1433/filebackup
```

3. Start and confirm the cutover took effect:

```
1 docker compose up -d --build --no-deps app
2 curl http://<new-ec2-ip>/api/v1/region-info
```

### 8.7 Phase 4 --- Verification Checklist

- ☐ S3 object count/byte size match, all three buckets (Phase 2 verify output)
- ☐ Row counts match for files, file\_tags, access\_logs
- ☐ /health → 200; /api/v1/region-info reports target-region values

- ❑ Full smoke test per category: upload → list → get → download → delete → 404 (Section 5.6)
- ❑ A few *pre-existing* files from each bucket download correctly – proves copied data, not just new writes, is intact

## 8.8 Phase 5 --- Traffic Cutover

Point the client-facing address at the new EC2 instance. Keep the source-region stack running, untouched, for a safety window.

## 8.9 Phase 6 --- Decommission the Source Region

After  $\geq 7$  days of the target serving real traffic without incident: take a final RDS snapshot, then delete the source RDS instance, EC2 instance, and – last – the source S3 buckets.

### i Note

**Rollback:** Phases 0–1 never touch the source stack, so rollback before Phase 5 is simply not cutting over. After Phase 5 but before Phase 6, rollback is a `.env` revert on the still-intact source instance plus re-pointing traffic – why Phase 6 stays last and manual.

## 9 Migration Automation Scripts

Full source for all three is in `scripts/` in the repository; idempotent where it matters, syntax-validated before inclusion in this project.

| Script                                 | What it does                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rds_region_migration.py</code>   | Snapshots the source RDS instance, copies the snapshot cross-region, restores a new instance, prints the endpoint. Engine-agnostic, non-destructive to the source. Args: <code>--source-region</code> , <code>--target-region</code> , <code>--db-instance-identifier</code> , <code>--target-db-instance-identifier</code> , <code>--db-instance-class</code> , <code>--subnet-group</code> , <code>--security-group-ids</code> , <code>--kms-key-id</code> |
| <code>s3_region_migration.sh</code>    | Single-bucket <code>aws s3 sync</code> primitive: <code>create-dest/warm/final/verify</code> modes                                                                                                                                                                                                                                                                                                                                                           |
| <code>s3_migrate_all_buckets.sh</code> | Runs the script above across <code>documents/media/archives</code> in parallel (background jobs + <code>wait</code> )                                                                                                                                                                                                                                                                                                                                        |

## 10 Security Notes

- **IAM instance role**, not static keys; S3 policy scoped to the three bucket ARNs only, never `s3:*`.
- **Presigned URLs only** (3600s expiry); RDS never public; soft deletes keep an audit trail; public access block + free SSE-S3 on all buckets.
- **Container hardening** (Section 6): no shell/package manager, non-root UID, read-only rootfs, capabilities dropped.
- **Shell-env precedence:** Compose prefers exported shell vars over `.env` – run `env | grep AWS_` first on shared machines.

## 11 Summary

A working, tested backend – three S3 buckets, RDS SQL Server, a hardened non-root container on the smallest practical EC2 instance, Swagger on port 80 – with region config isolated to one file, so migration is a data-replication/config exercise, not a rewrite. The bill is dominated by the SQL Server license ( $\approx$ \$70–90/mo; Section 3), the real cost lever. Practice Phases 0–4 of Section 8 freely against Sydney; nothing touches Singapore until Phase 6.